Workshop Task 4 10.21

Part 1

In leader_election scenario, all 3 nodes were initially followers. Node B timed out first and became a candidate in term 1. The node B as a candidate then sent reqeustvote to both A and C. After both other nodes had voted, node B was elected as the leader. In Raft, a leader must be elected first to simplify reasoning and prevent multiple leaders from sending conflicting AppendEntries. A majority is required to ensure the availability of the leader to followers, as well as to prevent multiple leaders being elected simultaneously. The role of terms in the election process is to track leaders and older elections.

Part 2

Leader B sent empty AppendEntries(term=1, entries=[]) heartbeats to A and C. Two heartbeat rounds were received by both followers. Their simulated election-timer reset counters increased, while B retained leadership. This shows why regular leader communication reduces unnecessary elections. Because election timers are not scheduled against real time, the experiment demonstrates the mechanism rather than measuring actual timing behaviour.

Part 3

B initially held leadership in term 1, but was then marked offline. A simulated timeout caused A to start term 2, receiving two of the three available votes. A was elected as the replacement leader. When B recovered, it received a term-2 heartbeat and remained a follower. The result illustrates how terms allow nodes to recognise newer authority and discard obsolete leadership assumptions.

Part 4

Leader B received the client command instances=10, created a log entry at index 1 in term 1, and sent it to A and C. Both followers acknowledged the entry. The simulator reported 3/3 acknowledgements and committed index 1. Every node ended with the same log entry and commit index. The critical sequence was client command, leader append, follower replication, acknowledgement, quorum decision, and notification of the commit index. This is logical commitment within an in-memory educational model; it is not proof of physical durability across real machine crashes.

Part 5

Leader B received the client command instances=10, created a log entry at index 1 in term 1, and sent it to A and C. Both followers acknowledged the entry. The simulator reported 3/3 acknowledgements and committed index 1. Every node ended with the same log entry and commit index. The critical sequence was client command, leader append, follower replication, acknowledgement, quorum decision, and notification of the commit index. This is logical commitment within an in-memory educational model; it is not proof of physical durability across real machine crashes.

Part 6

The five-node cluster initially elected A. Nodes C, D and E then failed, leaving only A and B online. The quorum requirement remained three. A could append an entry locally and replicate it to B, but 2/5 acknowledgements did not meet the threshold, so the entry was not committed. B also failed to obtain enough votes in a new election. This illustrates the safety-versus-availability choice: a system should refuse to commit new history when it cannot form a majority rather than incorrectly treat a minority as authoritative.

Part 7

A five-node cluster was divided into {A, B} and {C, D, E}. Before the partition, A was the term-1 leader.

Part 8

Leader A initially replicated two commands to follower B. B was then taken offline while A and C committed three more commands.

Part 9

The revised hardening.js scenario deliberately creates misconfiguration and failure events. An election timeout of 50 ms with a heartbeat interval of 30 ms triggers an unsafe-configuration warning under the educational rule that the former should be at least three times the latter. Repeated election attempts trigger election-storm detection. Three recorded changes of leader trigger leadership-churn detection. A five-node cluster reduced to two online nodes triggers a no-reachable-majority warning. After additional

writes, offline follower C is two entries behind the leader and triggers a lag warning. A deliberately corrupted commit index triggers an invalid-state warning.